SYSTEM DESIGN / INTERVIEW PLAYBOOK

The Complete Guide to HLD Data Structures

Bloom filters, HyperLogLog, count-min sketches, Merkle trees, tries, inverted indexes, geohashes, vector clocks and CRDTs — the structures interviewers expect you to name, size and defend.

Membership without the data

Bloom and cuckoo filters: the sizing formula, the false-positive trade, where they already run.

Counting at scale

HyperLogLog for distincts, count-min for frequency, t-digest for the p99 you report.

Finding things fast

Skip lists, inverted indexes, tries for typeahead, geohash · S2 · quadtree for proximity.

Agreeing without a leader

Merkle trees, vector clocks, CRDTs, SimHash — the structures distributed stores are built from.

7 parts · 4 diagrams · 14 interview Q&A · one-page cheat sheet

Revised August 2026

CONTENTS

What's inside

Every structure here exists because exactness was too expensive. Learn each one as a trade — what you gave up, what it costs in bytes — and you can pick one under pressure rather than recite names.

Part of the HLD concept series. Caching, Redis, databases and sharding are separate volumes — the structures here are referenced from all of them.

How to use this guide

Understand

Each structure gets the mechanism in plain English, then the number that makes it worth using.

Size it

Every entry carries bytes per item or total memory. Quoting the size is what makes the mention credible.

Place it

Each one ends with the real systems that use it, so you can cite rather than speculate.

HOW TO INTRODUCE ONE IN AN INTERVIEW

Never lead with the name. Lead with the problem, then the structure, then the cost: *"most of these lookups are for IDs that do not exist, so I would gate them with a Bloom filter — about 1.2 GB for a billion IDs at a 1% false-positive rate, and false positives only cost me one wasted cache lookup."* Name, number, trade-off. In that order, every time.

THE ONE THING TO INTERNALISE

All of these structures buy **memory or latency** by giving up something specific: exactness, deletability, or the ability to enumerate what is inside them. **If you cannot say which of the three you gave up, you have not chosen the structure — you have named it.**

Why approximate structures win

PART 1

A hash set of a billion 16-byte IDs is 16 GB before overhead, and roughly 50–90 GB with it. The same question — "have I seen this?" — costs 1.2 GB with a Bloom filter. That ratio is the whole reason this volume exists.

1.1 The four questions

Can I tolerate a wrong answer, and in which direction?

A Bloom filter never says "absent" about something present. That asymmetry is usually what makes it safe.

Do I need to enumerate, or only to ask?

Sketches answer questions about a set they cannot list. If you need the members back, you need the real data.

Must it merge?

HLL and count-min merge cleanly, so per-shard sketches roll up. A plain set does too, at full cost.

Does anything get deleted?

Classic Bloom filters cannot delete. Counting Bloom and cuckoo filters can, at extra bytes per item.

1.2 The trade-offs, side by side

Structure	Answers	Memory	Error
Hash set	exact membership, and enumerate	~16–90 bytes/item	none
Bloom filter	probably present / definitely absent	~1.2 bytes/item at 1%	false positives only
Cuckoo filter	same, plus delete	~1.5 bytes/item at 1%	false positives only
HyperLogLog	how many distinct	12 KB total	~0.81% standard error
Count-min sketch	how often did X occur	fixed grid, e.g. 1 MB	over-counts, never under
t-digest	percentiles	a few KB per stream	accurate in the tails
Reservoir sample	a uniform sample of a stream	k items	sampling error

The sentence that ties it together: **"I do not need the data, I need an answer about the data — so I will pay bytes for the answer instead of storing the set."**

Membership: Bloom and cuckoo filters

PART 2

The most name-dropped structure in system design, and the one candidates most often get half right. The half that matters is which way the error goes.

2.1 The mechanism, in four lines

- A bit array of **m** bits, all zero, and **k** independent hash functions.
- **Add:** hash the item k ways, set those k bits.
- **Query:** hash the item k ways. If any bit is 0 it is **definitely absent**. If all are 1 it is **probably present**.
- **Never delete:** clearing a bit could erase another item that shares it — which is why the classic filter is add-only.

`contains("carol")`

one of its three bits is 0

DEFINITELY NOT PRESENT — no false negatives, which is why it is safe to reject on.

`contains("bob")`

all three bits are 1

PROBABLY PRESENT — or three other keys lit those bits. That is the false positive.

Sizing, the only formula worth memorising

`m = −n ln(p) / (ln 2)^2` `k = (m/n) ln 2`

1% false positives cost ~9.6 bits per item, whatever the item is: 1.2 GB for a billion keys.

Figure 2.1 — *One bit array, three hashes, two kinds of answer. The asymmetry — no false negatives — is what makes it safe to reject on.*

2.2 Sizing it out loud

$$m = -n \cdot \ln(p) / (\ln 2)^2 \quad \cdot \quad k = (m/n) \cdot \ln 2$$

At a 1% false-positive rate that is ~9.6 bits per item and about 7 hash functions, whatever the items are. Ten million items ≈ 12 MB. A billion ≈ 1.2 GB. Halving p costs roughly one more byte per item, not double the memory.

Items	p = 1%	p = 0.1%	Exact hash set (16-byte ids)
1 million	1.2 MB	1.8 MB	~50–90 MB
100 million	120 MB	180 MB	~5–9 GB
1 billion	1.2 GB	1.8 GB	~50–90 GB

2.3 The variants worth naming

Counting Bloom filter

Replace each bit with a 4-bit counter so deletes decrement instead of clearing. ~4× the memory.

Cuckoo filter

Stores small fingerprints in a cuckoo hash table: supports delete, better cache locality, and beats Bloom below ~3% error. Insertions can fail when it is nearly full.

Scalable / layered Bloom

Chain filters of growing size when you do not know n in advance — the honest answer to "what if the set grows?".

Quotient filter

Mergeable and resizable, used in some LSM engines. Worth one sentence if the interviewer digs.

2.4 Where it already runs

- **Cache penetration gate:** hold every valid ID; reject lookups for IDs that cannot exist before they touch cache or database.
- **LSM storage engines:** one filter per SSTable, so a read skips files that certainly do not hold the key — this is what makes LSM reads survivable.
- **Web crawlers:** "have I fetched this URL?" over billions of URLs, where a rare false positive means one page skipped.
- **Chrome / Safe Browsing and Bigtable:** the canonical citations if you want one name-drop rather than three.
- **Redis: BF.ADD** / **BF.EXISTS** in the Bloom module, so you get it without running new infrastructure.

Cardinality: HyperLogLog and friends

PART 3

"How many unique users watched this video?" asked for every video, every hour, for ten years. Exact answers need the set; you almost never need exact.

3.1 The mechanism

Hash each item to a uniformly random bit string and watch for improbable patterns: a hash with ten leading zeros turns up about once every 1,024 distinct items, so seeing one implies roughly a thousand distinct items. One estimate is far too noisy, so the hash is split into 16,384 registers, each tracking its own maximum, and the harmonic mean of the registers becomes the estimate.

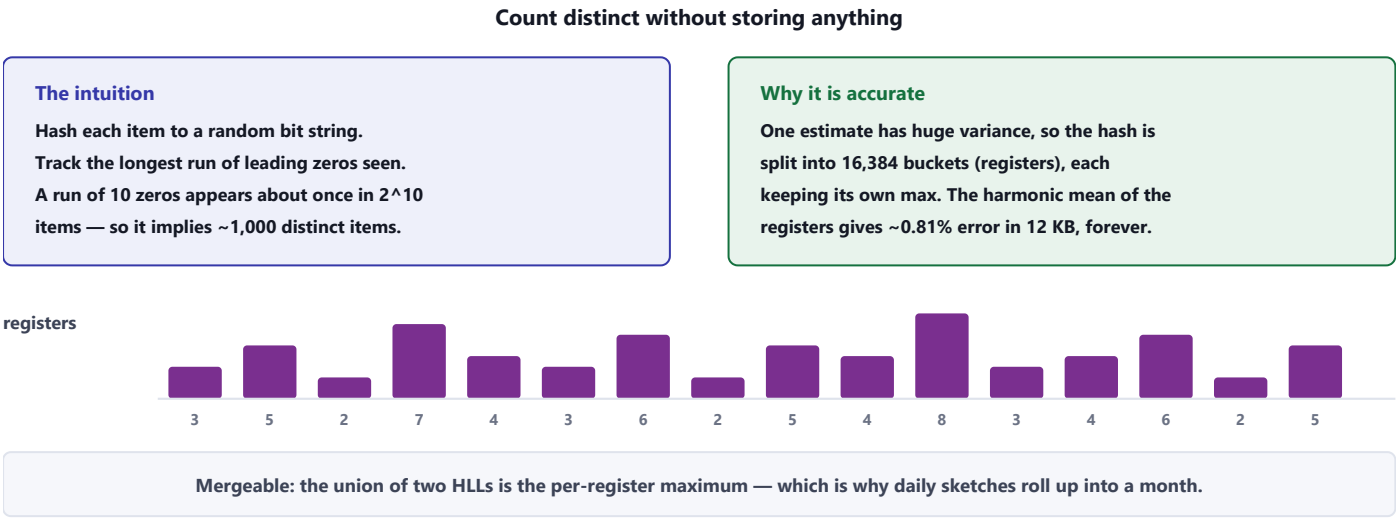

Figure 3.1 — *The two halves of HyperLogLog: an improbability counter, then averaging over thousands of independent copies to kill the variance.*

The numbers to quote: **12 KB, ~0.81% standard error, unlimited cardinality, and unions are free** — merging two sketches is the per-register maximum, so daily sketches roll into a month and per-shard sketches roll into a global figure.

3.2 When not to use it

Small cardinalities

Below a few thousand items a plain set is smaller *and* exact. Real implementations know this and switch representation (HLL++ uses a sparse encoding first) — worth mentioning that the library already handles it.

When the answer must be exact

Billing, compliance, anything a regulator reads. An 0.8% error on a revenue number is not a design trade-off, it is a bug.

3.3 MinHash and Jaccard similarity

Different question, same trick. To estimate how much two sets overlap, keep only the smallest k hash values of each set: the fraction of shared minima estimates the Jaccard similarity. It is how near-duplicate detection, recommendation candidate generation and plagiarism checks work at scale, and it composes with locality-sensitive hashing to find similar pairs without comparing every pair.

3.4 The family, one line each

Structure	Question it answers	The number
HyperLogLog	How many distinct items?	12 KB, 0.81% error, mergeable
HLL++	Same, accurate on small sets too	Sparse below ~thousands, then dense
Linear counting	Distincts when cardinality is small	Simpler, worse at scale
MinHash	How similar are two sets?	k hashes per set; error falls with k
SimHash	Are these two documents near-identical?	64-bit fingerprint, Hamming distance
Theta sketch	Distincts with set operations (intersections too)	Bigger than HLL, does more

Frequency, top-K and percentiles

PART 4

Three questions that look similar and need three different structures: how often did this happen, what are the busiest keys, and what is my p99.

4.1 Count-min sketch

A grid of **d** rows × **w** counters. To add an event, hash the key once per row and increment that row's counter. To query, take the **minimum** across the rows — collisions can only inflate a counter, so the smallest of d readings is the closest to the truth. It never undercounts, which is exactly the guarantee you want when hunting abuse or hot keys.

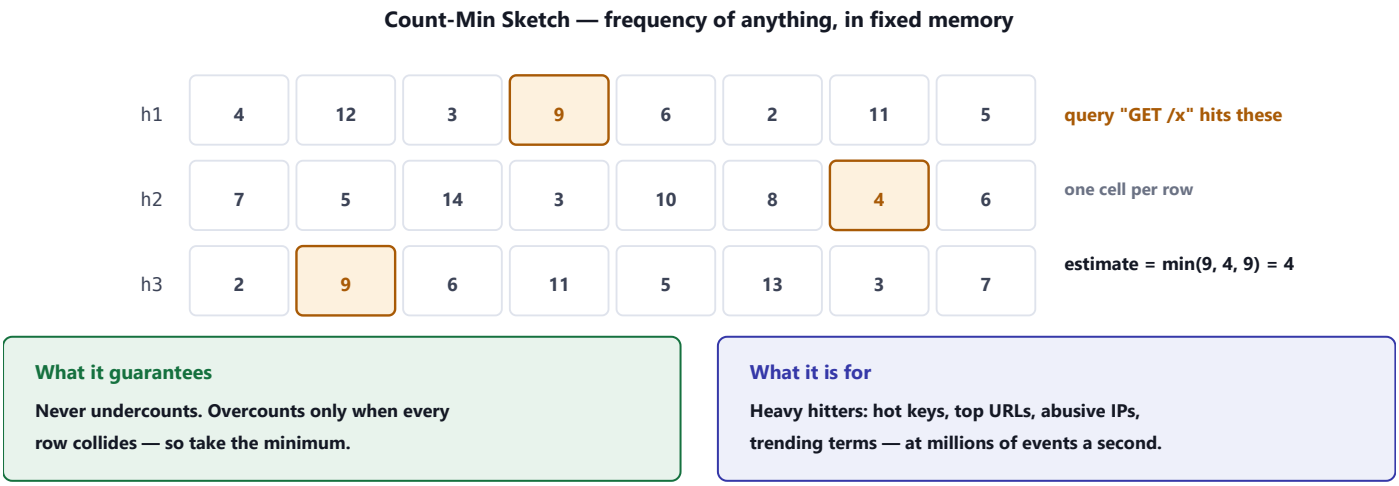

Figure 4.1 — *Count-min sketch: constant memory regardless of how many distinct keys arrive, and an error bound you can state.*

Interview framing: **"I would find hot keys with a count-min sketch in the proxy — fixed memory, no per-key allocation, never undercounts — and keep an exact top-K alongside it with Space-Saving."**

4.2 Top-K without keeping every key

Space-Saving / Misra-Gries

Keep k counters; a new key evicts the current minimum and inherits its count. Finds heavy hitters in one pass, memory O(k).

Count-min + heap

Sketch for counts, small heap for the current leaders. The standard trending-topics design.

Redis in practice

A sorted set with **ZINCRBY** when the key space is bounded; **TOPK** and **CMS** commands from the Bloom module when it is not.

The honest caveat

Approximate top-K can miss a key that is just below the threshold. Say so — it is usually fine for trending, never fine for billing.

4.3 Percentiles, and the mistake everyone makes

> **YOU CANNOT AVERAGE PERCENTILES**
>
> The mean of ten servers' p99 values is not the fleet p99. To aggregate you need a mergeable summary of the distribution — an HDR histogram (fixed buckets, exact within bucket width) or a **t-digest** (adaptive, far more accurate in the tails, and mergeable). Saying this out loud in an observability discussion is a reliable senior signal.

4.4 Reservoir sampling

To keep k uniformly random items from a stream of unknown length: keep the first k, then for item n replace a random one with probability k/n. One pass, O(k) memory, and every item equally likely. It is how you sample traces for debugging, or logs for analysis, without deciding the sample rate in advance.

Finding things: indexes and ordering

PART 5

Search, autocomplete and leaderboards are three of the most common design prompts, and each one is a structure question wearing a product costume.

5.1 Skip list — the sorted structure you already use

A linked list with extra express lanes: each node appears in higher levels with probability one half, so search skips ahead and lands in O(log n). No rebalancing, and range scans are a walk along the bottom level. This is what backs a Redis sorted set, which is why **ZREVRANK** can give an exact rank among fifty million players in O(log n) — the query SQL cannot do without scanning.

5.2 Inverted index — how search actually works

Documents are tokenised, normalised (lowercase, stem, strip stop words) and stored as a map from term to a sorted **posting list** of document ids. A query intersects the posting lists of its terms, then ranks the survivors — classically by BM25, now often blended with vector similarity.

Why it is fast

Posting lists are sorted and delta-compressed, so intersection is a merge, and skip pointers jump ahead.

Why writes are batched

Segments are immutable and merged in the background — which is why Elasticsearch is near-real-time, not real-time.

Typo tolerance

Character n-grams or edit-distance automata; “fuzzy” is a different index, not a flag on the same one.

The interview point

An inverted index is a second copy of your data, so it needs a feed (CDC or an outbox) and a reindex plan.

5.3 Trie and radix tree — autocomplete

A prefix tree: each edge a character, so “find everything starting with tes” is a walk to that node. A radix tree compresses single-child chains, which is what makes it memory-viable. The trick that makes typeahead fast is **precomputing the top-K completions at every node**, so a keystroke is a lookup rather than a subtree scan, and the whole structure is rebuilt offline from query logs.

Distributed-systems structures

PART 6

These are the structures that let independent machines agree, or at least reconcile, without a coordinator on the hot path.

6.1 Merkle trees

Hash each block of data, then hash pairs of hashes upward to a single root. Two replicas compare roots: equal means identical, different means descend into the differing subtree. You find the one bad block in log(n) comparisons instead of streaming everything.

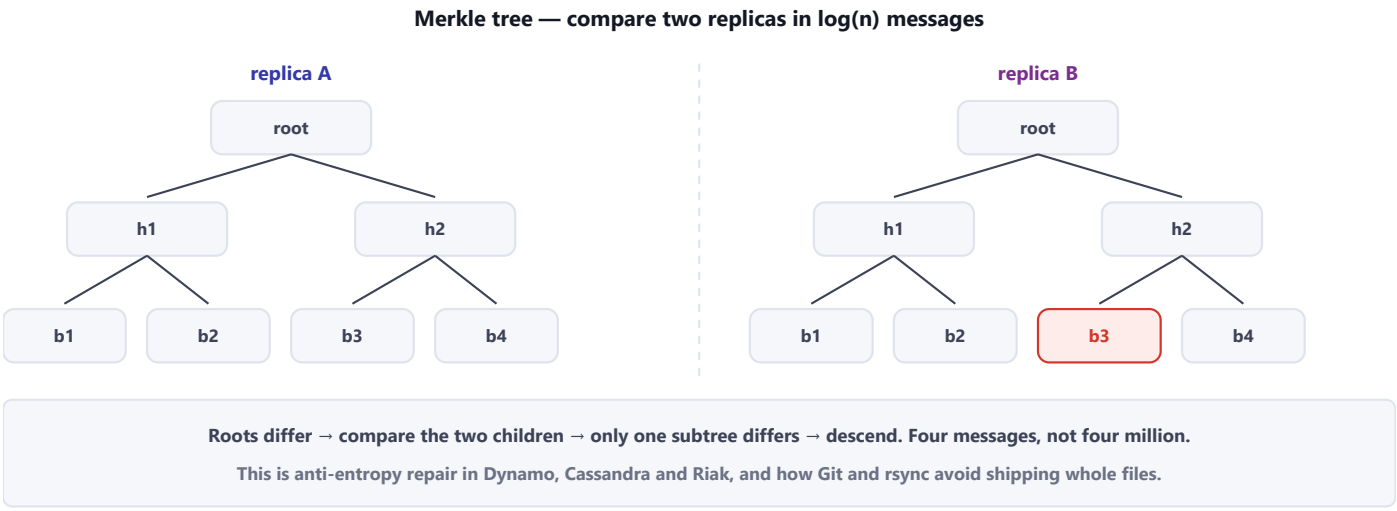

Figure 6.1 — *Anti-entropy repair in three steps. Dynamo-style stores, Git and rsync all lean on exactly this property.*

6.2 Clocks: Lamport, vector, and why timestamps lie

Mechanism	What it gives you	What it costs
Wall clock (LWW)	A simple “latest wins” rule	Clock skew silently discards writes — correct only when you accept losing one
Lamport clock	A total order consistent with causality	Cannot tell concurrent from ordered — one counter, no context
Vector clock	Detects concurrency: “these two writes are siblings”	Size grows with the number of writers; needs pruning
Version vector / dotted	The production form used by Dynamo-style stores	Conflicts are surfaced to the application to resolve

6.3 CRDTs — merging without asking

A conflict-free replicated data type is a structure whose merge function is commutative, associative and idempotent, so replicas that receive the same updates in any order converge to the same value. No coordination, no conflicts to resolve, and the cost is that the merge rule must be decided in advance — and it is not always the rule the product wants.

Type	Merge rule	Used for
G-counter	Per-replica counters, sum them	Increment-only metrics, likes
PN-counter	Two G-counters, subtract	Counters that go down too
LWW register	Highest timestamp wins	Profile fields, settings
OR-set	Adds and removes with unique tags	Shopping carts, tag sets
RGA / Yjs, Automerge	Ordered sequence with unique ids	Collaborative text editing

6.4 Geospatial indexing

Geohash

Interleave latitude and longitude bits into a string, so a shared prefix means physical proximity. Simple, sortable, and it has edge cases at cell boundaries.

S2 / H3

Google’s spherical cells and Uber’s hexagons: uniform cell areas, hierarchical, and no boundary pathologies. The modern answer.

Quadtree

Recursively split space until each cell holds few points — good for skewed density, and it is the classic interview answer for “find nearby drivers”.

R-tree / PostGIS

Bounding-box index for shapes and polygons, not just points. What a real GIS database uses.

6.5 Coordination-free IDs

Snowflake: 41 bits of millisecond timestamp, 10 bits of machine id, 12 bits of sequence — 4,096 ids per millisecond per machine, sortable by time, no coordination on the write path. UUIDv7 is the same idea in the UUID format. Both exist because auto-increment needs a single authority, and a single authority does not shard.

The interview itself

PART 7

These structures are worth points only when they arrive as answers to a problem you have already stated. Named unprompted, they read as flashcards.

7.1 The three-beat mention

1
The problem
"Most of these lookups are for IDs that never existed"

→

2
The structure
"A Bloom filter in front of the cache"

→

3
The number
"1.2 GB for a billion IDs at 1%, and a false positive costs one wasted lookup"

7.2 Matching problem to structure

When you hear	Reach for	Because
"Has this URL been crawled?"	Bloom filter	Billions of items, and a rare false positive only skips one page
"How many unique viewers?"	HyperLogLog	12 KB per counter and the sketches merge across shards and days
"Which keys are hot right now?"	Count-min + Space-Saving	Fixed memory at millions of events a second, never undercounts
"Show the p99 across the fleet"	t-digest / HDR histogram	Percentiles do not average; distributions merge
"Rank among 50 million players"	Skip list (sorted set)	O(log n) exact rank, which SQL cannot do without a scan
"Search these documents"	Inverted index	Term → posting list intersection, then ranking
"Autocomplete as I type"	Trie with precomputed top-K	A keystroke becomes a lookup instead of a scan
"Find drivers near me"	S2 / H3 / quadtree	Proximity needs spatial locality in the key itself
"Do these replicas match?"	Merkle tree	log(n) comparisons instead of streaming the data set
"Offline edits must merge"	CRDT	A merge rule that converges without coordination

7.3 Fourteen questions with model answers

Q1 Explain a Bloom filter and its failure mode.

A bit array plus k hash functions. Adding sets k bits; a query that finds any bit zero proves absence, and all-ones means probably present. So the only error is a false positive — never a false negative — which is why it is safe to *reject* on and never safe to *confirm* on. About 9.6 bits per item gives 1%, and the classic filter cannot delete because clearing a bit could erase another item.

Q2 How big a Bloom filter for a billion IDs, and what does the error cost you?

~1.2 GB at 1% false positives, or ~1.8 GB at 0.1%, with about seven hash functions. In a cache-penetration gate a false positive means one lookup that would have been skipped still happens — a wasted read, not a wrong answer. Compare that with a hash set of the same IDs at 50 GB or more.

Q3 Count the unique visitors to a billion pages, hourly.

One HyperLogLog per page per hour: 12 KB each, ~0.8% error, and unions are the per-register maximum so hours roll up into days and shards roll up into a global number. I would say explicitly that this is an estimate, and keep an exact count only for the handful of pages someone bills against.

Q4 Find the top 100 hottest keys in a 5-million-QPS stream.

A count-min sketch for frequencies plus Space-Saving for the leaders: fixed memory, no per-key allocation, and the sketch never undercounts so a hot key cannot hide. Per shard, then merge the sketches. The caveat I would volunteer is that a key just below the threshold can be missed, which is fine for hot-key mitigation and not fine for billing.

Q5 Why can’t you average p99 across servers?

Because a percentile is a property of a distribution, not a number you can mean. Ten servers each reporting p99 = 100 ms could have a fleet p99 far higher, depending on traffic share and shape. You need mergeable summaries: HDR histograms with fixed buckets, or t-digests, which stay accurate in the tails and merge exactly.

Q6 Design autocomplete for a search box.

A trie or radix tree over the query corpus with the **top-K completions precomputed at every node**, so a keystroke is one lookup. Build it offline from query logs, ship it as an immutable artefact, and keep it in memory — it is small enough. Add a personalisation pass on top of the shared candidates, cache the prefix results, and debounce on the client. Typos are a separate n-gram or edit-distance index, not a flag.

Q7 How would you detect near-duplicate documents at scale?

SimHash or MinHash. SimHash gives each document a 64-bit fingerprint where similar documents differ in only a few bits, so “near-duplicate” becomes a Hamming-distance lookup. MinHash estimates Jaccard similarity from the smallest k hashes, and with locality-sensitive hashing you only compare documents that land in the same bucket — that is what makes it linear instead of quadratic.

Q8 Two replicas may have diverged. How do you find the difference cheaply?

Merkle trees. Each replica hashes its key ranges into a tree; comparing roots is one message, and if they differ you descend only into the subtrees that disagree — log(n) messages to locate the bad range, then repair just that range. This is anti-entropy in Dynamo-style stores, and the same trick Git and rsync use.

Q9 What is a vector clock and when would you use one?

A per-replica counter set attached to a value, so two versions can be compared: one dominates the other, or they are concurrent siblings. You use it when writes can happen in more than one place and last-write-wins is not acceptable, because it lets you *detect* the conflict instead of silently dropping a write. The cost is size growth with the number of writers and the need to resolve siblings in application code.

Q10 When would you use a CRDT instead of a lock?

When replicas must accept writes while partitioned and then converge — collaborative editing, offline-capable apps, per-region counters. A CRDT's merge is commutative, associative and idempotent, so order does not matter and no coordination is needed. The price is that the merge rule is fixed in advance: a G-counter cannot express "reject if it would go negative", so inventory still wants a transaction.

Q11 Find all drivers within 2 km. What is the index?

Spatial cells: H3 or S2, or a quadtree if density is very uneven. Each driver's position becomes a cell id, drivers are stored by cell, and a proximity query reads the covering cell plus its neighbours — because the target can sit near a boundary. Geohash works the same way with a string prefix and is fine to name, as long as you mention the boundary problem.

Q12 How does a Redis sorted set give exact ranks so fast?

It is a skip list plus a hash map. The skip list keeps members ordered by score with probabilistic express lanes, so insert, delete and rank are O(log n); the hash map gives O(1) score lookup by member. That combination is why leaderboards, sliding-window rate limiters and priority queues all end up on a sorted set.

Q13 Generate unique IDs across 100 servers without coordination.

Snowflake: 41 bits of millisecond timestamp, 10 bits of machine id, 12 bits of sequence — time-ordered, 8 bytes, and 4,096 per millisecond per machine. UUIDv7 is the same shape in UUID clothing. The operational details worth naming are machine-id assignment and what you do when the clock steps backwards: refuse to mint rather than risk duplicates.

Q14 The interviewer asks "why not just use a hash set?"

Sometimes that is the right answer, and saying so is a good signal: below a few million items, exact and simple beats clever. The switch happens when the set no longer fits in memory on one node, or when you need it in a hot path on every server, or when it must merge across shards. Then I trade exactness for bytes — and I say which direction the error goes.

7.4 Red flags interviewers listen for

Saying this	Says this about you
"Bloom filter" with no size or error rate	Name-dropping; the number is the whole point
"Bloom filters can have false negatives"	Backwards — and it inverts where they are safe to use
Deleting from a classic Bloom filter	Would corrupt other items; you want counting or cuckoo
HyperLogLog for a number someone bills on	0.8% error is not a rounding difference to finance
Averaging p99 across instances	Percentiles do not average; you need mergeable histograms
A sketch where a set of 10,000 would do	Complexity with no payoff
"Use a trie" with no top-K precomputation	Then every keystroke scans a subtree
Geohash with no boundary handling	Nearby points can sit in different cells
Last-write-wins presented as conflict resolution	It is conflict *deletion*; say which write you are losing
Naming a structure before naming the problem	Flashcards, not design

7.5 Real systems worth name-dropping

Google Bigtable & Chrome

Bloom filters per SSTable to skip files on read, and in Chrome's Safe Browsing to check URLs locally before asking the server. The two canonical citations.

Redis probabilistic commands

PFADD/PFCOUNT for HyperLogLog, BF.* for Bloom, CMS.* and TOPK.* for sketches — evidence that these are production tools, not textbook curiosities.

Dynamo & Cassandra

Merkle-tree anti-entropy, vector clocks for sibling detection, consistent hashing for placement. One paper, three structures.

Uber H3 · Google S2

Hierarchical spatial indexing in the open, and the reason "geohash" is no longer the best answer to a proximity question.

REVISION

One-page cheat sheet

The night-before page. The numbers are the part that makes a mention credible.

The sizes to quote

Structure	Memory	Accuracy	The one-liner
Bloom filter	~9.6 bits/item at 1%	no false negatives	"probably present, definitely absent"
Cuckoo filter	~12 bits/item at 1%	no false negatives	"Bloom, but deletable"
HyperLogLog	12 KB fixed	~0.81% error	"count distinct, mergeable, forever"
Count-min sketch	grid, e.g. 4×64k	never undercounts	"frequency in fixed memory"
Space-Saving	k counters	exact for real heavy hitters	"top-K in one pass"
t-digest	a few KB	accurate in the tails	"percentiles that merge"
Skip list	O(n), ~1.33 pointers/node	exact	"ordered, O(log n) rank"
Merkle tree	one hash per block	exact	"diff two replicas in log(n)"

Pick one in ten seconds

- **Membership at scale** → Bloom (or cuckoo if you delete).
- **Distinct count** → HyperLogLog. **Frequency** → count-min. **Top-K** → Space-Saving.
- **Percentiles** → t-digest or HDR histogram, never an average of percentiles.
- **Ranking / ordering** → skip list. **Prefix search** → trie with top-K. **Full text** → inverted index.
- **Proximity** → S2/H3/quadtree. **Replica repair** → Merkle. **Concurrent writes** → vector clocks, then CRDTs.

Eight facts

The error has a direction

Bloom: false positives only. Count-min: over-counts only. Say which way it is wrong.

Sketches cannot enumerate

They answer questions about a set they cannot list. Need the members? Keep the data.

Mergeability is the superpower

HLL, count-min and t-digest merge, so per-shard and per-hour sketches roll up.

Deletion is expensive

Classic Bloom cannot delete. Counting Bloom pays ~4×; cuckoo pays a little.

12 KB and 0.81%

The two HyperLogLog numbers. Quote them and the mention becomes credible.

Percentiles never average

Merge distributions, not summaries. This one is worth a whole sentence out loud.

Precompute at the node

A trie is only fast for typeahead when each node already knows its top completions.

Merkle turns compare into descend

Equal roots mean identical data; unequal roots mean one path to walk.

Where each one already runs

Structure	In production
Bloom filter	LSM engines (per-SSTable), crawlers, Chrome Safe Browsing, Redis BF.*
HyperLogLog	Redis PFADD, BigQuery APPROX_COUNT_DISTINCT, Presto, analytics pipelines
Count-min	Hot-key detection in proxies, abuse pipelines, Redis CMS.*
t-digest	Elasticsearch percentiles, Prometheus/HDR-style latency reporting
Skip list	Redis sorted sets, LevelDB/RocksDB memtables
Inverted index	Elasticsearch/OpenSearch, Lucene, every search box you use
Merkle tree	Dynamo, Cassandra, Riak repair; Git; blockchains
CRDT	Yjs and Automerge in collaborative editors; Riak data types

IF YOU SAY NOTHING ELSE, SAY THIS

"Every one of these structures trades exactness, deletability or enumerability for memory. So I name the problem first, then the structure, then the number and the direction of the error: a Bloom filter at 9.6 bits an item that can only be wrong by saying yes; a HyperLogLog at 12 KB and 0.8%; a count-min sketch that never undercounts. If I cannot say what I gave up, I have not chosen the structure — I have just named it."

Sources & further reading — Bloom (1970), *Space/time trade-offs in hash coding with allowable errors*; Fan et al., *Cuckoo Filter* (2014); Flajolet et al., *HyperLogLog* (2007) and Heule et al., *HyperLogLog in Practice* (2013); Cormode & Muthukrishnan, *Count-Min Sketch* (2005); Metwally et al., *Space-Saving* (2005); Dunning & Ertl, *t-digest*; Broder, *MinHash* and Charikar, *SimHash*; Pugh, *Skip Lists* (1990); Shapiro et al., *Conflict-free Replicated Data Types* (2011); DeCandia et al., *Dynamo* (SOSP 2007); Google S2 and Uber H3 documentation; the Redis probabilistic module docs; Lucene's index-format documentation.